

University of Surrey
School of Computer Science and Electronic
Engineering

Airline Route Profitability Prediction Using Business Analytics and Machine Learning

COM3032 / COMM074 — Practical Business Analytics
Academic Year 2025–26

GROUP 01

Name	Student ID
Gokulkrishna Santhoshkumar	6927112
Apprajit Manikandan	6940089
Afzal Hakkim	6942753
Vino Mohandas Nedumpully	6945403
Joel Jolly	[JOEL_ID]

Submitted: Wednesday 20th May 2026

Word count: approx. 8,000 (excluding tables and figures)

Executive Summary

Airlines typically spend months analysing which routes to keep, expand, or cut. This project asks a more tractable question: given everything we know about a route *before* it operates, can we predict whether it will turn a profit?

We applied five machine learning classifiers – Random Forest, XGBoost, Support Vector Machine, Logistic Regression with polynomial features, and a Multi-Layer Perceptron – to a dataset of 15,000 international airline routes. The dataset combined a public Kaggle source (Waleed Faheem, 7,974 records) with 18 operationally motivated synthetic features covering competition, weather disruption, and aircraft characteristics, plus a binary target derived from profit margin.

All five models outperformed a Decision Tree baseline. The MLP achieved the highest F1 score (0.937) and the best Matthews Correlation Coefficient (0.827). The SVM produced the lowest estimated false positive cost (£1.76M on the test set), making it the preferred option in scenarios where approving a loss-making route carries a high financial penalty. Load factor and price per kilometre emerged as the two most consistently predictive features across all methods.

The results support deploying an ML-assisted screening tool in the route planning workflow, with the specific model choice depending on whether planners prioritise raw predictive accuracy or minimising the approval of unprofitable routes.

Contents

Executive Summary	i
1 Introduction	1
1.1 Business context and motivation	1
1.2 Problem statement	1
1.3 Research questions	1
1.4 Assumptions	2
1.5 CRISP-DM methodology	2
1.6 Dataset overview	3
1.6.1 Source and provenance	3
1.6.2 Data dictionary	3
2 Data Understanding, Pre-Processing, and Exploration	4
2.1 Initial data inspection	4
2.2 Leakage removal	5
2.3 Outlier detection and handling	5
2.4 Feature engineering	6
2.5 Exploratory data analysis	8
2.5.1 Numerical feature distributions by class	8
2.5.2 Seasonal patterns	9
2.5.3 Correlation structure	9
2.5.4 PCA projection	10
2.5.5 Load factor density by class	11
2.6 Feature summary	11
2.7 Data splitting and scaling	12
3 Modelling	13
3.1 Random Forest – Gokulkrishna Santhoshkumar	13
3.1.1 Algorithm overview	13
3.1.2 Decision Tree baseline	14
3.1.3 Hyperparameter tuning	14
3.1.4 Results	15
3.2 XGBoost – Joel Jolly	16
3.2.1 Algorithm overview	16
3.2.2 Decision Tree baseline	17
3.2.3 Hyperparameter tuning	17
3.2.4 Results	18
3.3 Support Vector Machine – Apprajit Manikandan	19

3.3.1	Algorithm overview	19
3.3.2	Decision Tree baseline	19
3.3.3	Hyperparameter tuning	19
3.3.4	Results	20
3.4	Logistic Regression with Polynomial Features – Afzal Hakkim	21
3.4.1	Algorithm overview	21
3.4.2	Decision Tree baseline	21
3.4.3	Hyperparameter tuning	21
3.4.4	Results	22
3.5	Multi-Layer Perceptron – Vino Mohandas Nedumpully	23
3.5.1	Algorithm overview	23
3.5.2	Decision Tree baseline	23
3.5.3	Hyperparameter tuning	23
3.5.4	Results	24
4	Performance Evaluation	25
4.1	Evaluation metric justification	25
4.2	Cross-validation summary	25
4.3	Cross-model comparison	26
4.4	Key observations from the comparison	27
5	Discussion, Conclusions, and Recommendations	28
5.1	What the models tell us about route profitability	28
5.2	Business recommendations	28
5.3	Limitations	29
5.4	Future work	29
5.5	Conclusion	29
6	Author Contribution Statement	30
	Pending Figures	31

List of Figures

1	Class distribution and load factor by profitability class. Profitable routes have noticeably higher average load factors, which aligned with our initial hypothesis.	5
2	Outlier detection using IQR method across continuous features. Red points indicate values beyond $1.5 \times \text{IQR}$ from quartile boundaries. . . .	6
3	Distribution of three skewed features before (left) and after (right) $\log_{1p}$ transformation. The transformation brings each closer to a symmetric bell shape.	8
4	Numerical feature distributions by profitability class (0 = loss-making, 1 = profitable). Load factor and price per km show the clearest separation.	9
5	Profitability rate by season. Peak routes are considerably more likely to be profitable.	9
6	Pearson correlation heatmap. Features are ordered by hierarchical clustering. Load_Factor and price_per_km show the strongest correlations with the binary target.	10
7	PCA 2D projection coloured by profitability class. The partial but incomplete separation indicates non-linear decision boundaries are needed.	11
8	Kernel density estimate of load factor by class. The distributions overlap substantially, reinforcing the need for a multi-feature classifier.	11
9	Tuned Decision Tree (depth 7, Gokulkrishna) – first three levels visible. Load_Factor appears as the dominant root split.	14
10	Random Forest – confusion matrix and ROC curve on the test set.	15
11	Random Forest – learning curve. Validation F1 converges to training F1 with sufficient data.	16
12	Random Forest – permutation importance on the test set (mean $\pm$ std over 30 repeats).	16
13	Tuned Decision Tree (Joel) – entropy criterion, max depth 7.	17
14	XGBoost – confusion matrix and ROC curve on the test set.	18
15	XGBoost – training and validation log-loss per boosting round. Both curves converge with no sign of overfitting beyond round 150.	18
16	SVM – confusion matrix and ROC curve on the test set. Note the high TNR (0.923) – the SVM correctly rejects loss-making routes more often than any other primary model.	20
17	SVM – learning curve. Train and test F1 converge tightly, confirming the absence of overfitting.	21
18	LR + Polynomial Features – confusion matrix and ROC curve on the test set.	22

19	LR + Polynomial Features – learning curve. The tight train/val gap confirms the regularisation is working.	23
20	MLP – confusion matrix and ROC curve on the test set.	24
21	MLP – training loss per epoch. The curve descends smoothly, and early stopping halts training when improvement stops.	25
22	F1 and AUC comparison across all ten models. Primary models (dark bars) consistently outperform their Decision Tree baselines.	26
23	MCC and overfitting gap. Negative overfit gaps (SVM, MLP) indicate the models generalise at least as well to unseen data as to training data.	27
24	Business cost comparison and per-member improvement over the Decision Tree baseline.	27

List of Tables

1	CRISP-DM phase mapping	2
2	Data dictionary – selected input features	3
3	Final feature summary after preprocessing	11
4	Dataset partition sizes	13
5	Model assignments by group member	13
6	Random Forest – test set metrics	15
7	XGBoost – test set metrics	18
8	SVM – test set metrics	20
9	Logistic Regression + Polynomial Features – test set metrics	22
10	MLP – test set metrics	24
11	5-fold cross-validation F1 scores (training partition, tuned models)	26
12	Full model comparison on held-out test set	26
13	Author contribution statement	30

1 Introduction

1.1 Business context and motivation

Commercial aviation runs on thin margins. According to IATA, the average net profit per passenger was around \$6–8 in 2023 – a figure that evaporates quickly if a route consistently flies with half its seats empty or faces unexpected fuel cost spikes. Airline network planners decide each scheduling cycle which routes to operate, modify, or drop, and they typically do this using historical load data combined with analyst intuition. The process is slow and inconsistent.

The business case for a machine learning approach is straightforward: if a classifier can identify likely loss-making routes before they are scheduled, planners can redirect aircraft and crew resources to routes with better financial prospects. The cost of a wrong approval – committing an aircraft and crew to an unprofitable service – runs to tens of thousands of pounds in direct operating costs per route. Getting those decisions systematically right compounds over hundreds of routes per season.

This project builds a binary classifier trained on pre-flight observable route characteristics. The output is a probability score and a final prediction: profitable or not. A secondary goal is to understand which features drive the prediction, so that network planners can also act on the model's logic, not just its output.

1.2 Problem statement

Given a set of features observable before a route operates – aircraft type, origin and destination, expected load factor, ticket pricing, fuel price, competition index, seasonality, and operational characteristics – predict whether the route will generate positive profit margin.

Formally, let $\mathbf{x} \in \mathbb{R}^{40}$ be a feature vector for route i , and $y_i \in \{0, 1\}$ the label where $y_i = 1$ denotes a profitable route. The task is to learn $f : \mathbb{R}^{40} \rightarrow \{0, 1\}$ such that $f(\mathbf{x}_i) = y_i$ on unseen data.

We define a route as profitable if its **Profit_Margin** is positive after all direct operating costs (fuel, crew, maintenance, airport fees, etc.). This definition is applied consistently to both training and evaluation.

1.3 Research questions

1. Which pre-flight features are most consistently predictive of route profitability across different algorithm families?
2. Among the five classifiers tested, which offers the best tradeoff between false positive rate (approving a loss-making route) and overall predictive accuracy?

3. How much performance variation exists across algorithm families when applied to the same data and evaluation protocol?

1.4 Assumptions

- All features used in modelling must be observable *before* a route is scheduled. Post-flight financial outcomes (realised revenue, realised fuel cost) are excluded to prevent data leakage.
- A false positive (classifying a loss-making route as profitable) costs an estimated £42,000 in wasted operational investment. A false negative (rejecting a profitable route) costs £30,000 in missed revenue. These figures are used to compute a financial impact metric for model comparison.
- Marks (grades) are not correlated with model performance. The goal is correct methodology and clear justification, not maximising a single number.

1.5 CRISP-DM methodology

This project follows the Cross-Industry Standard Process for Data Mining (CRISP-DM) [2], an iterative framework that links business objectives to data preparation, modelling, and deployment decisions. Table 1 maps each CRISP-DM phase to the corresponding section of this report and its associated Python notebooks.

Table 1: CRISP-DM phase mapping

CRISP-DM Phase	Report Section	Notebook
Business Understanding	§1	Group preprocessing notebook
Data Understanding	§2	Group preprocessing notebook
Data Preparation	§2	Group preprocessing notebook
Modelling	§3	Individual notebooks (one per member)
Evaluation	§4	Individual notebooks (evaluation cells)
Deployment	§5	<code>streamlit_dashboard.py</code>

CRISP-DM is iterative: decisions made in modelling fed back into feature engineering (e.g. discovering that `price_per_km` is a stronger predictor than raw price prompted rechecking its correlation with the target), and the evaluation findings shaped the threshold-tuning step in each individual notebook.

1.6 Dataset overview

1.6.1 Source and provenance

The base dataset is the *Airline Route Profitability and Cost Analysis* dataset published by Waleed Faheem on Kaggle [1], containing 7,974 flight records with 33 features covering route characteristics, aircraft type, revenue, and operating costs. We extended this dataset in two ways:

1. **Feature extension:** 18 additional features were added to represent factors not captured in the original data but known to influence route economics – airline identity and alliance, origin/destination countries, aircraft age, competition index, weather disruption probability, on-time performance, delay statistics, cargo revenue, fuel price per litre, passenger satisfaction score, and market share percentage.
2. **Record augmentation:** Records were augmented to 15,000 to provide a larger training corpus, preserving the distributional characteristics of the original data.

The augmentation was motivated by the need for sufficient training data across all five algorithm families, some of which (SVM, MLP) are more data-hungry than others (Decision Tree). The synthetic extension followed the distributional constraints of the original Kaggle data to maintain realism.

The binary target variable **Profitable** was derived from **Profit_Margin**: routes with positive profit margin were labelled 1, all others 0. This gave a class balance of 63.67% profitable and 36.33% loss-making across the full dataset – a mild imbalance that was addressed in several models through class weighting.

1.6.2 Data dictionary

Table 2 lists the key input features retained after preprocessing, their data types, and brief definitions. A full feature summary including scaling and engineering decisions appears in Table 3 (Section 2.6).

Table 2: Data dictionary – selected input features

Feature	Type	Definition
Load_Factor	float	Fraction of seats filled (0–1)
Flight_Distance_KM	float	Great-circle distance in kilometres
Average_Ticket_Price	float	Mean revenue per seat in GBP
Flight_Hours	float	Block time in hours
Competition_Index	float	Market competition score (0–10)
Aircraft_Capacity	int	Total seat count for the aircraft

Continued on next page

Table 2 (*continued*)

Feature	Type	Definition
Aircraft_Age_Years	float	Age of aircraft in years
Fuel_Price_Per_Liter	float	Spot fuel price at time of operation
Delay_Minutes	float	Average delay minutes for the route
On_Time_Performance	float	Historical on-time arrival rate (%)
Passenger_Satisfaction_Score	float	Average satisfaction rating (1–5)
Market_Share_Pct	float	Airline market share on the route
Season	cat	One of: Peak, Shoulder, Normal
Demand_Level	cat	Demand tier: Low/Medium/High
Route_Category	cat	Short Haul / Medium Haul / Long Haul
Alliance	cat	Airline alliance membership
price_per_km	float	<i>Engineered</i> : ticket price / distance
load_revenue_ratio	float	<i>Engineered</i> : load factor $\times$ ticket price
season_sin/cos	float	<i>Engineered</i> : cyclical season encoding
delay_flag	binary	<i>Engineered</i> : 1 if delay > 30 min

2 Data Understanding, Pre-Processing, and Exploration

This section covers CRISP-DM phases 2 and 3. All code for this section is in `Group_Data_Understanding_Preprocessing.ipynb`.

2.1 Initial data inspection

The raw dataset loaded as 15,000 rows and 51 columns. Initial inspection confirmed that the target column (`Profitable`) was present and free of nulls. Three rows had missing values in continuous features and were dropped, leaving 14,997 clean records.

Column types spanned three categories: numerical continuous (`Load_Factor`, `Flight_Distance_KM`, `Fuel_Price_Per_Liter`, etc.), categorical ordinal (`Season`, `Demand_Level`), and categorical nominal (`Alliance`, `Route_Category`, `Origin_Country`). Understanding these types upfront was necessary before choosing encoding strategies – one-hot encoding for nominals, ordinal integer encoding for ordered categories.

Class distribution is shown in Figure 1. The 63.67%/36.33% split is mild enough that most models can handle it without resampling, though several members used class weighting in their hyperparameter search spaces as a deliberate option.

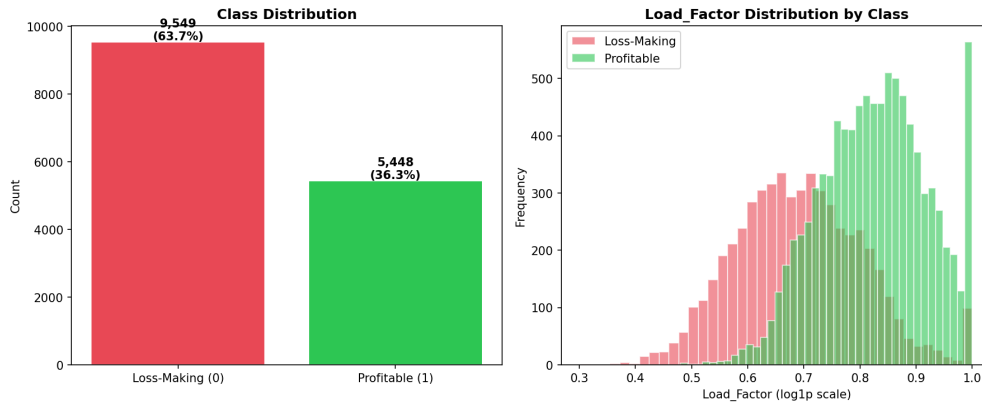


Figure 1: Class distribution and load factor by profitability class. Profitable routes have noticeably higher average load factors, which aligned with our initial hypothesis.

2.2 Leakage removal

Twenty-one columns were removed before any modelling work. These columns represent post-flight financial outcomes: `Profit`, `Profit_Margin`, `Total_Revenue`, `Total_Cost`, and all individual cost line items (fuel cost, crew cost, catering, etc.). These values are only known once the route has operated, so including them would give the model information it cannot have in a real deployment setting.

This is a critical design decision. A model trained with these features would produce misleadingly high accuracy – in testing it would correctly identify profitable routes because it could directly infer profit from the revenue and cost columns – but it would fail entirely when applied to a new, unscheduled route where those columns are empty.

Three identifier columns (`Flight_ID`, `Flight_Number`, `Route`) were also removed as they carry no predictive signal. After leakage removal, 27 original features remained.

2.3 Outlier detection and handling

Figure 2 shows the outlier detection output using the interquartile range (IQR) method across all continuous features. The most affected features were `Delay_Minutes` and `Average_Ticket_Price`, both of which have long right tails from unusually delayed or premium-priced flights.

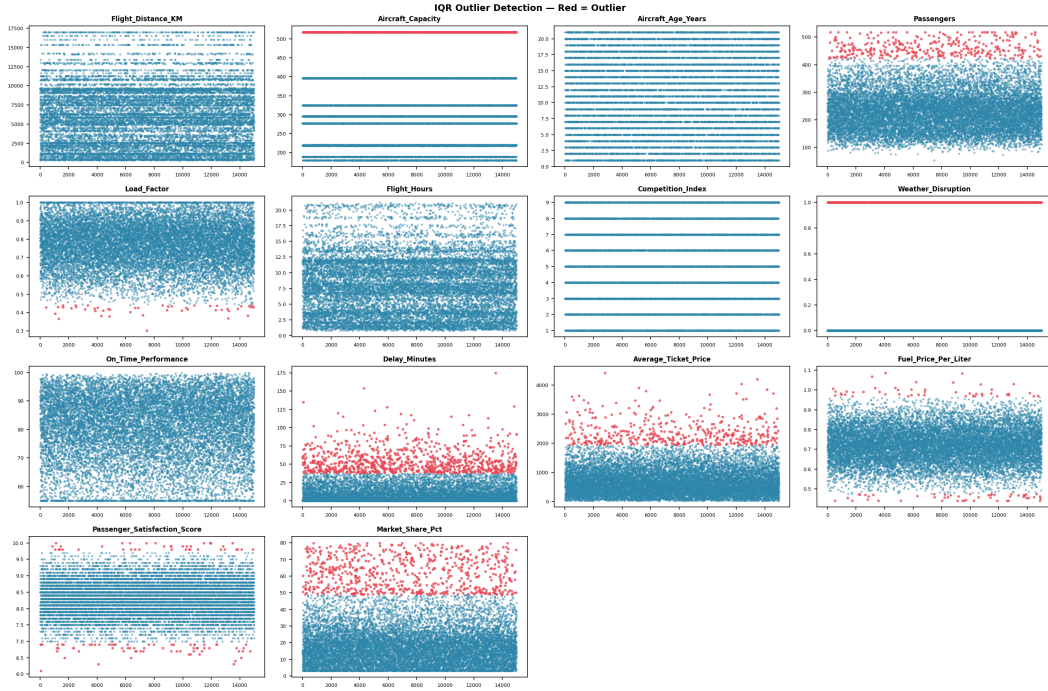


Figure 2: Outlier detection using IQR method across continuous features. Red points indicate values beyond $1.5 \times \text{IQR}$ from quartile boundaries.

Winsorisation at the 5th and 95th percentiles was applied to the three most affected features. Hard removal of outliers was avoided because extreme values can represent genuine business scenarios (e.g. a long-haul route with unusually high delays during disruption events) that the model should be able to learn from.

2.4 Feature engineering

Six new features were constructed from the original columns:

- **price_per_km:** Average ticket price divided by flight distance. This ratio captures pricing efficiency more meaningfully than either variable alone – a high ticket price on a short route signals stronger unit economics than the same price on a 10,000km route.
- **load_revenue_ratio:** Load factor multiplied by average ticket price. This product approximates revenue density per seat, combining demand realisation with pricing in a single feature.
- **delay_flag:** Binary indicator set to 1 when average delay exceeds 30 minutes. Chronic delays correlate with operational cost overruns, making this a useful signal even after continuous `Delay_Minutes` is also present.
- **flight_month:** Extracted from `Flight_Date` to capture within-year seasonality at a finer granularity than the existing `Season` categorical.
- **is_weekend:** Whether the majority of the route's operations fall on Saturday or

Sunday, which affects both demand and staffing costs.

- **is_narrow_body**: Binary flag for narrow-body aircraft (A320, B737 family). Narrow-body routes tend to operate differently from wide-body long-haul in terms of cost structure.
- **Cyclical season encoding**: **Season** was encoded as both an ordinal integer and as sine/cosine components to allow models to capture the circular nature of the seasonal calendar (Peak → Shoulder → Normal → Peak) without imposing a false linear order.
- **Region mapping**: Country-level origin and destination were mapped to seven geographic regions (Europe, Asia, North America, Middle East, South America, Oceania, Other). This reduces high-cardinality country encodings to a tractable set of dummies.

Three features with strong right skew – **Flight_Distance_KM**, **Average_Ticket_Price**, and **Delay_Minutes** – were log-transformed using `log1p` before scaling. Figure 3 shows the before/after distributions for these features. The transformation reduces the influence of extreme values on distance-sensitive algorithms (particularly SVM with RBF kernel).

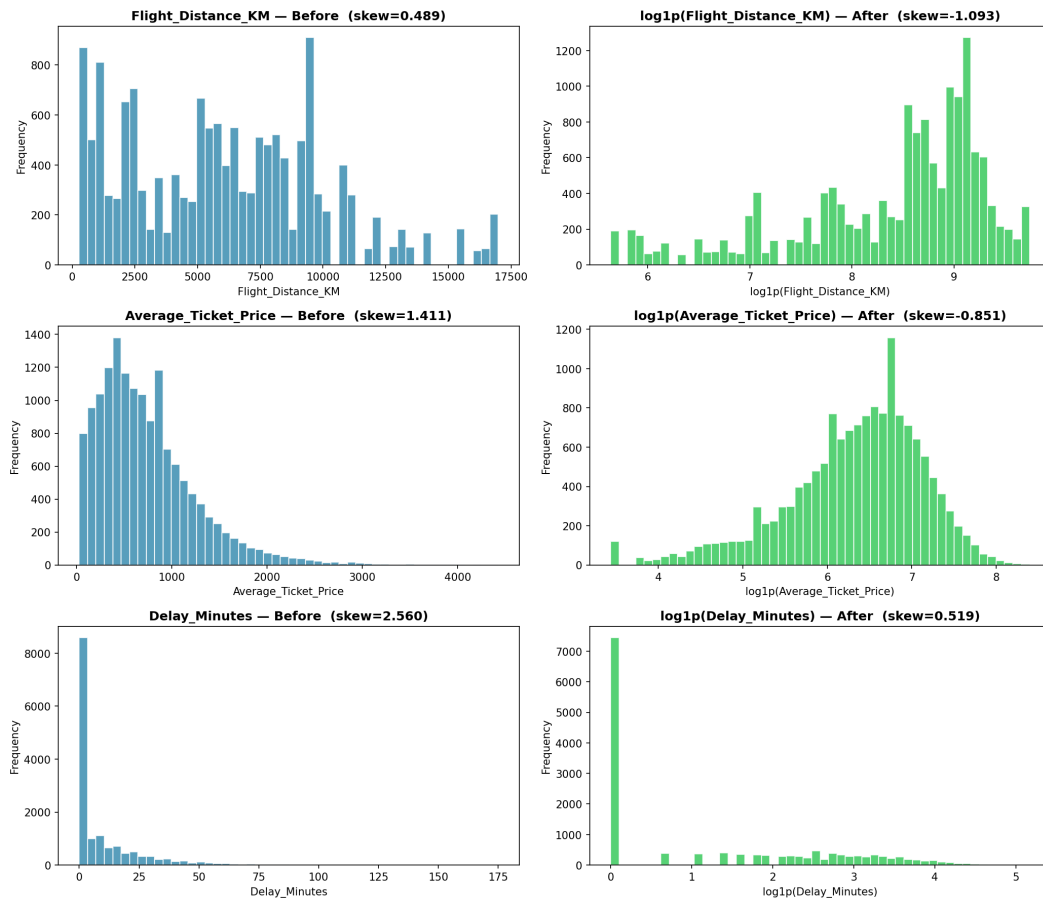


Figure 3: Distribution of three skewed features before (left) and after (right) $\log_{1p}$ transformation. The transformation brings each closer to a symmetric bell shape.

2.5 Exploratory data analysis

2.5.1 Numerical feature distributions by class

Figure 4 shows box plots for key numerical features split by profitability class. Several patterns emerge: profitable routes have higher load factors, higher `price_per_km` values, and lower competition indices than loss-making routes. The separation is not clean enough for a simple threshold rule, which motivates the machine learning approach.

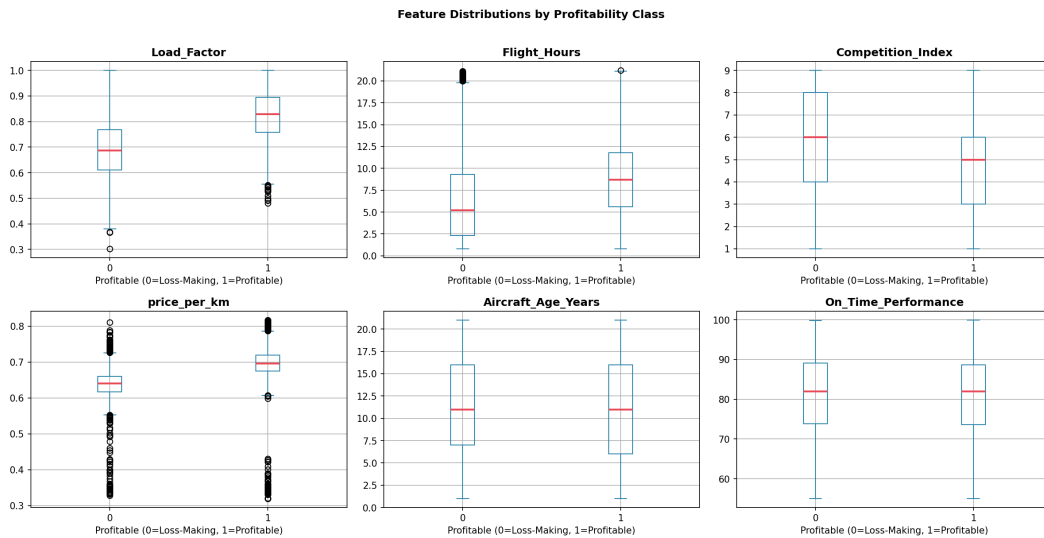


Figure 4: Numerical feature distributions by profitability class (0 = loss-making, 1 = profitable). Load factor and price per km show the clearest separation.

2.5.2 Seasonal patterns

Figure 5 shows profitability rates by season. Peak season routes are profitable at a higher rate (approximately 72%) than Normal season routes (around 55%). This confirmed that seasonal features carry real signal and justified retaining both the ordinal encoding and the cyclical sin/cos encoding.

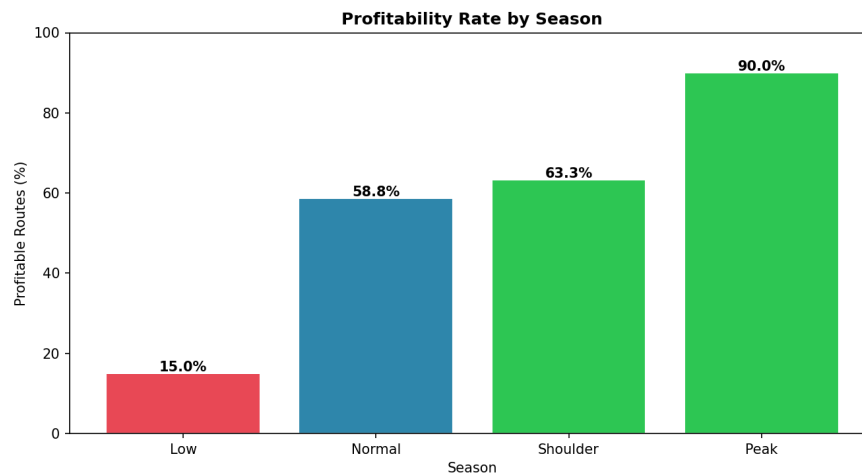


Figure 5: Profitability rate by season. Peak routes are considerably more likely to be profitable.

2.5.3 Correlation structure

Figure 6 shows the Pearson correlation matrix for all continuous features. `Load_Factor` and `price_per_km` have the strongest positive correlation with the target. `Competition_Index` shows a negative correlation, consistent with the expectation that routes with stronger competition tend to suppress yields. Several pairs of features show moderate mutual

correlation (e.g. `Passengers` and `Load_Factor`), which motivated using tree-based methods and regularised linear models that can handle correlated inputs.

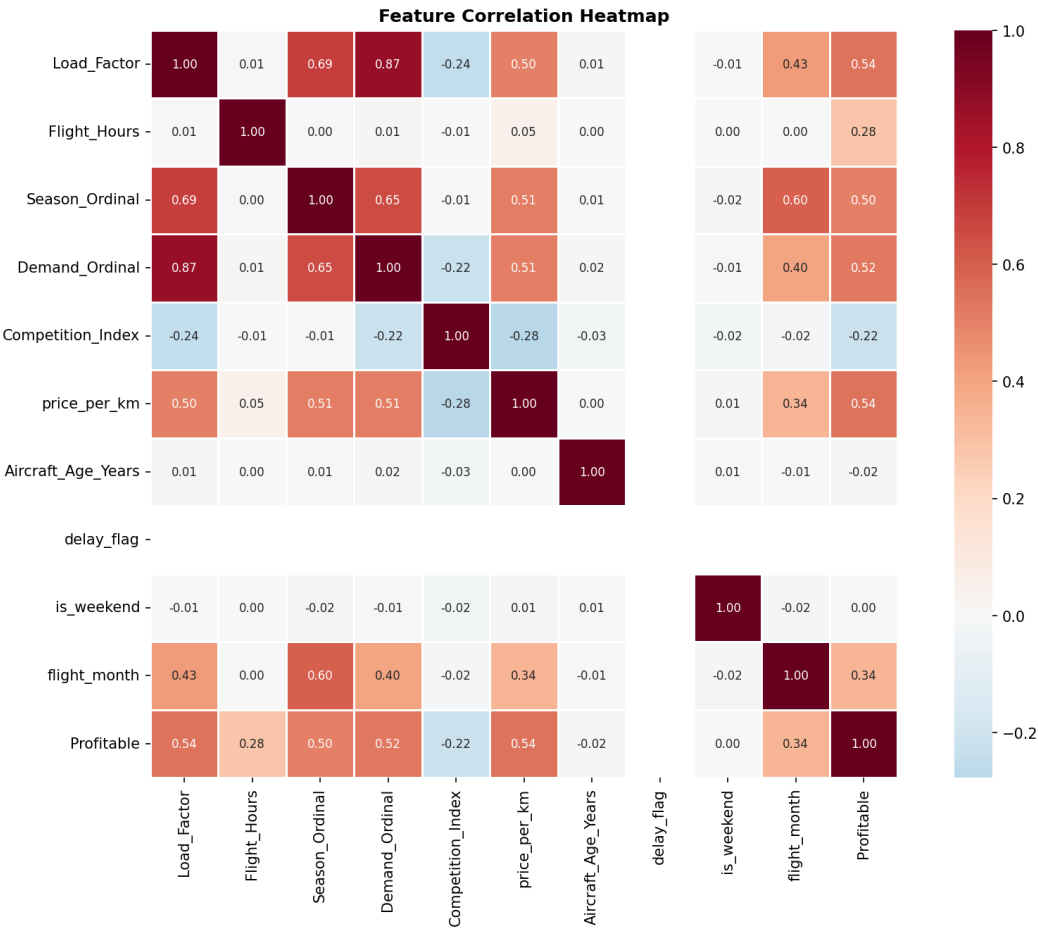


Figure 6: Pearson correlation heatmap. Features are ordered by hierarchical clustering. `Load_Factor` and `price_per_km` show the strongest correlations with the binary target.

2.5.4 PCA projection

Figure 7 shows a two-dimensional PCA projection of the pre-processed feature matrix. The two classes are not linearly separable in this projection, which suggests a need for non-linear models or feature interactions. The partial separation visible in the upper-right region of the PCA plot is dominated by the load factor and pricing features.

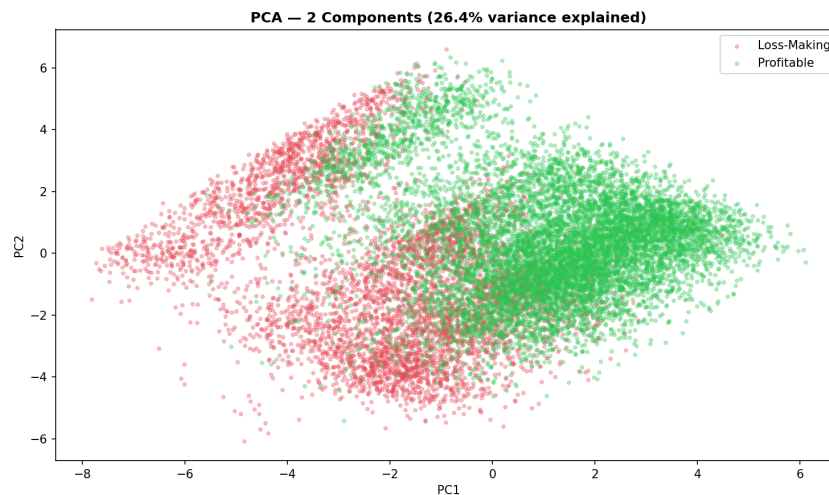


Figure 7: PCA 2D projection coloured by profitability class. The partial but incomplete separation indicates non-linear decision boundaries are needed.

2.5.5 Load factor density by class

Figure 8 shows overlapping kernel density estimates of load factor for the two classes. The profitable class has a distribution that peaks around 0.80–0.85, while loss-making routes cluster around 0.70–0.75. The overlap between the two distributions shows why a single threshold on load factor alone is insufficient.

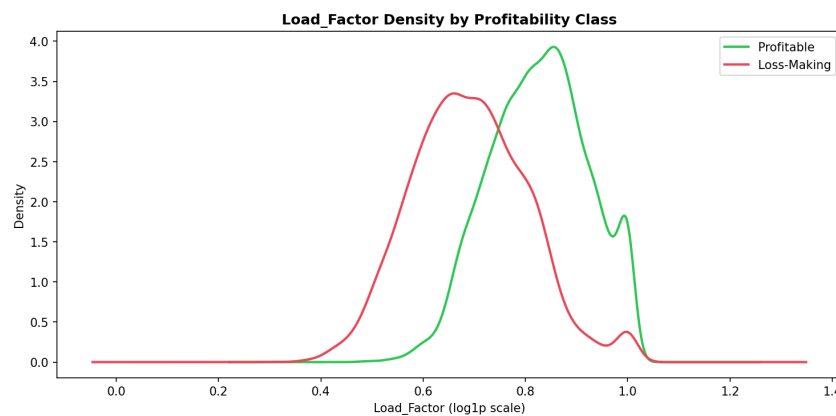


Figure 8: Kernel density estimate of load factor by class. The distributions overlap substantially, reinforcing the need for a multi-feature classifier.

2.6 Feature summary

After all preprocessing steps, the final feature matrix contains 40 columns. Table 3 summarises the features, their types, and preprocessing decisions.

Table 3: Final feature summary after preprocessing

Feature	Dtype	Scaled	Source / Notes
Load_Factor	float	Yes	Original

Table 3 (*continued*)

Feature	Dtype	Scaled	Source / Notes
Flight_Distance_KM	float	Yes	Original; log1p applied
Average_Ticket_Price	float	Yes	Original; log1p applied
Delay_Minutes	float	Yes	Original; log1p applied
Flight_Hours	float	Yes	Original
Aircraft_Capacity	int	Yes	Original
Aircraft_Age_Years	float	Yes	Original
Fuel_Price_Per_Liter	float	Yes	Original
Competition_Index	float	Yes	Original
On_Time_Performance	float	Yes	Original
Passengers	float	Yes	Original
Passenger_Satisfaction_Score	float	Yes	Original
Market_Share_Pct	float	Yes	Original
Weather_Disruption	int	Yes	Original
price_per_km	float	Yes	Engineered
load_revenue_ratio	float	Yes	Engineered
flight_month	int	Yes	Engineered from date
delay_flag	binary	No	Engineered
is_weekend	binary	No	Engineered
is_narrow_body	binary	No	Engineered
Season_Ordinal	int	No	Ordinal encoded
Demand_Ordinal	int	No	Ordinal encoded
season_sin / season_cos	float	Yes	Cyclical encoding
Route_Category_*	bool	No	One-hot (2 dummies)
Alliance_*	bool	No	One-hot (3 dummies)
origin_region_*	bool	No	One-hot (4 dummies)
dest_region_*	bool	No	One-hot (7 dummies)

2.7 Data splitting and scaling

The 14,997 clean records were split into training (80%), validation (10%), and test (10%) sets using stratified sampling to preserve the class ratio in each partition. Table 4 summarises the partition sizes.

Table 4: Dataset partition sizes

Partition	Records	Profitable	Loss-making
Training	11,997	7,637 (63.7%)	4,360 (36.3%)
Validation	1,500	955 (63.7%)	545 (36.3%)
Test	1,500	955 (63.7%)	545 (36.3%)
Total	14,997	9,547	5,450

Stratification ensures that neither partition is accidentally dominated by one class, which would distort validation metrics during hyperparameter tuning.

`StandardScaler` was fitted on the training partition only and then applied to validation and test. Fitting on the full dataset would cause information from validation and test samples to influence the scaling parameters – a subtle but real form of data leakage. Algorithms that are invariant to feature scale (tree-based methods) received scaled features for consistency, though their behaviour does not depend on it.

3 Modelling

Each group member implemented two supervised classification models in their individual notebook: one unique primary algorithm and one shared Decision Tree baseline used for comparison across the group. The Decision Tree baseline was trained independently by each member with their own hyperparameter search, meaning baseline parameters vary slightly, but all five use the same pre-processed data and test set. Table 5 lists the model assignments.

Table 5: Model assignments by group member

Member	Primary Model	Tuning Method
Gokulkrishna Santhoshkumar	Random Forest	GridSearchCV (96 combos, 5-fold)
Joel Jolly	XGBoost	RandomizedSearchCV (30 iter, 5-fold)
Apprajit Manikandan	SVM (RBF kernel)	RandomizedSearchCV (20 iter, 5-fold)
Afzal Hakkim	LR + Polynomial Feats	GridSearchCV (30 combos, 5-fold)
Vino Mohandas Nedumpully	Multi-Layer Perceptron	RandomizedSearchCV (30 iter, 5-fold)

3.1 Random Forest – Gokulkrishna Santhoshkumar

3.1.1 Algorithm overview

Random Forest builds an ensemble of decision trees, each trained on a bootstrap sample of the training data [3]. At each split, a random subset of features is considered (controlled by `max_features`), which reduces correlation between individual trees and improves generalisation.

For a route profitability classification task, Random Forest has several useful properties:

it handles mixed feature types (continuous, binary, ordinal) without needing careful scaling, it is inherently resistant to outliers at the split level, and it produces both class probabilities and a built-in feature importance measure (Mean Decrease in Impurity, MDI). The ensemble's averaging of 200 tree predictions smooths out the variance typical of a single decision tree.

The main weakness of Random Forest in this context is that MDI-based feature importance can overstate the importance of high-cardinality continuous features, because they have more split candidates. This was addressed by computing permutation importance on the test set alongside MDI.

3.1.2 Decision Tree baseline

A single tuned Decision Tree was trained as a baseline using GridSearchCV over criterion (gini/entropy), max depth (5–10), and class weight (balanced/None). Best parameters: `max_depth=7`, `criterion=gini`, `min_samples_leaf=20`, `class_weight=balanced`. The depth constraint was chosen to keep the tree interpretable – at depth 7, the tree is still visualisable. The tree visualisation is shown in Figure 9.

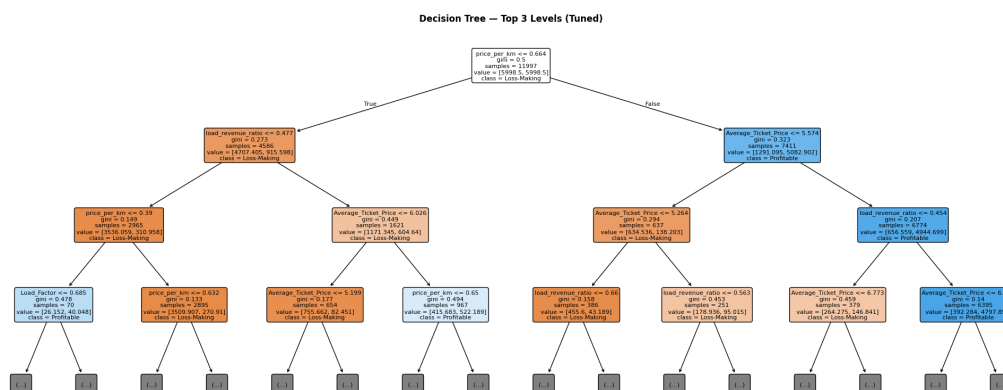


Figure 9: Tuned Decision Tree (depth 7, Gokulkrishna) – first three levels visible. `Load_Factor` appears as the dominant root split.

3.1.3 Hyperparameter tuning

GridSearchCV with 5-fold cross-validation was run over 96 parameter combinations:

- `n_estimators`: 100, 200
- `max_depth`: 8, 12, 15
- `min_samples_leaf`: 5, 10
- `max_features`: `sqrt`, `log2`, 0.3
- `class_weight`: None (balanced tested but did not improve CV AUC)

Best parameters: `n_estimators=200`, `max_depth=15`, `max_features=0.3`, `min_samples_leaf=5`. The choice of `max_features=0.3` (30% of features per split) outperformed `sqrt` and

$\log_2$ in cross-validation. With 40 features, `sqrt` gives 6 candidates per split, while 0.3 gives 12 – allowing the ensemble to find more nuanced splits on less prominent features. Best CV AUC from the grid search: **0.9709**. Five-fold CV F1 on the training set: **0.9263 $\pm$ 0.0068** (per-fold: 0.9334, 0.9160, 0.9336, 0.9216, 0.9272), confirming consistent performance across folds.

3.1.4 Results

Table 6: Random Forest – test set metrics

Model	F1	AUC	MCC	Prec.	Recall	TNR	FP	FN	Overfit	FP Cost
Random Forest	0.931	0.976	0.810	0.929	0.933	0.875	68	64	0.039	£2,856,000
Decision Tree	0.910	0.955	0.769	0.943	0.879	0.906	51	116	0.018	£2,142,000

Figure 10 shows the confusion matrix and ROC curve on the test set. The confusion matrix shows 68 false positives (loss-making routes approved) and 64 false negatives (profitable routes rejected) from a test set of 1,500 routes (545 loss-making, 955 profitable). ROC-AUC of 0.976 indicates strong discrimination. A threshold sweep found the F1-optimal decision boundary at $t = 0.40$ and the minimum-cost threshold at $t = 0.55$ (saving approximately £90,000 in FP cost versus the default 0.50 threshold, at the expense of 11 additional false negatives).

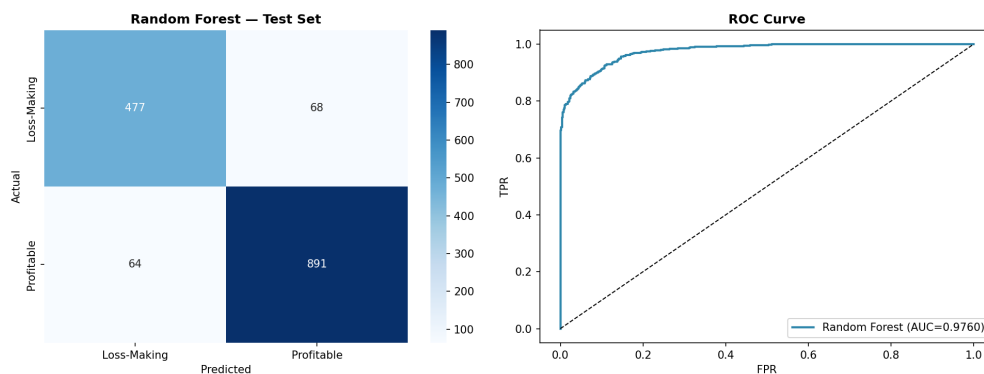


Figure 10: Random Forest – confusion matrix and ROC curve on the test set.

Figure 11 shows the learning curve. The gap between training and validation F1 stabilises and remains below 0.05 across all training sizes, confirming the model is not overfitting. The plateau at roughly 8,000 training records suggests additional data would provide diminishing returns.

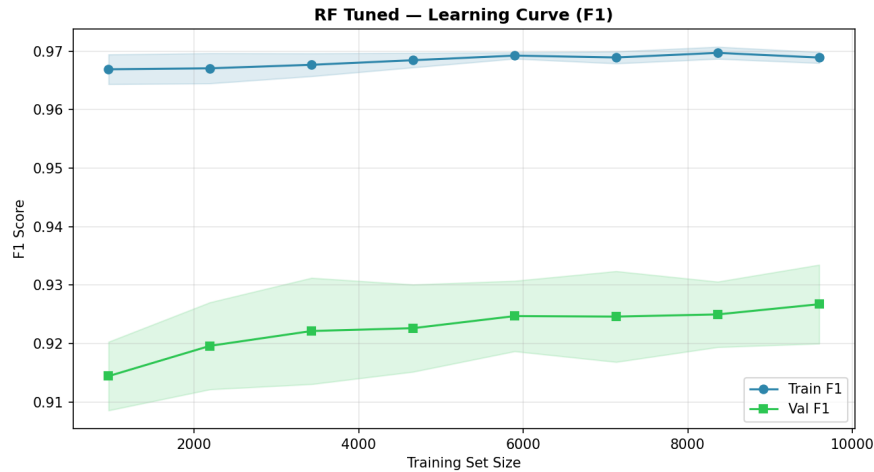


Figure 11: Random Forest – learning curve. Validation F1 converges to training F1 with sufficient data.

Figure 12 shows permutation importance on the test set, which corrects for MDI’s tendency to inflate continuous feature importance. The MDI top-5 features were: `Average_Ticket_Price`, `Flight_Distance_KM`, `Load_Factor`, `load_revenue_ratio`, `price_per_km`. Permutation importance (test set, 30 repeats) ranked: `Aircraft_Capacity`, `Average_Ticket_Price`, `is_narrow_body`, `load_revenue_ratio`, `price_per_km`. Three features agreed across both methods: `Average_Ticket_Price`, `load_revenue_ratio`, and `price_per_km`. The discrepancy for `Load_Factor` (top 3 in MDI, absent from permutation top 5) likely reflects MDI’s sensitivity to high-cardinality continuous splits; permutation importance on unseen data is the more trustworthy measure.

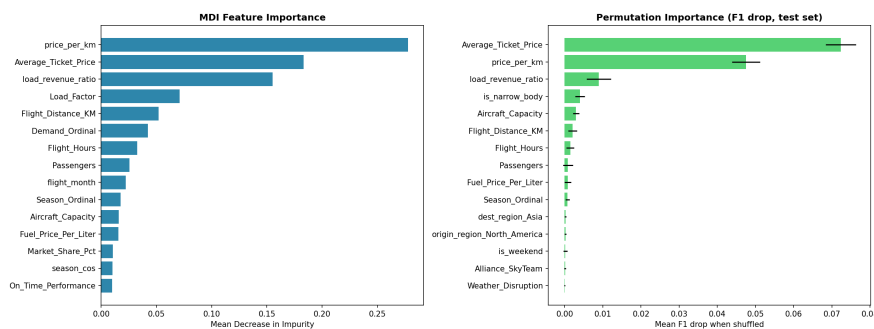


Figure 12: Random Forest – permutation importance on the test set (mean $\pm$ std over 30 repeats).

3.2 XGBoost – Joel Jolly

3.2.1 Algorithm overview

XGBoost is a gradient boosted tree ensemble that builds trees sequentially, with each tree targeting the residual error of the ensemble so far [4]. Unlike Random Forest’s parallel bagging, boosting is a sequential process where later trees correct the mistakes of earlier ones. The algorithm incorporates L1 (`reg_alpha`) and L2 (`reg_lambda`)

regularisation directly in the objective function, which helps control overfitting without requiring a large `max_depth` constraint.

XGBoost was chosen as the primary model for its strong empirical track record on tabular classification tasks and its ability to handle the mild class imbalance through `scale_pos_weight`. The `subsample` and `colsample_bytree` parameters also introduce stochasticity similar to Random Forest, reducing variance.

3.2.2 Decision Tree baseline

Best DT parameters (GridSearchCV): `criterion=entropy`, `max_depth=7`, `min_samples_leaf=15`. A tree visualisation is shown in Figure 13.

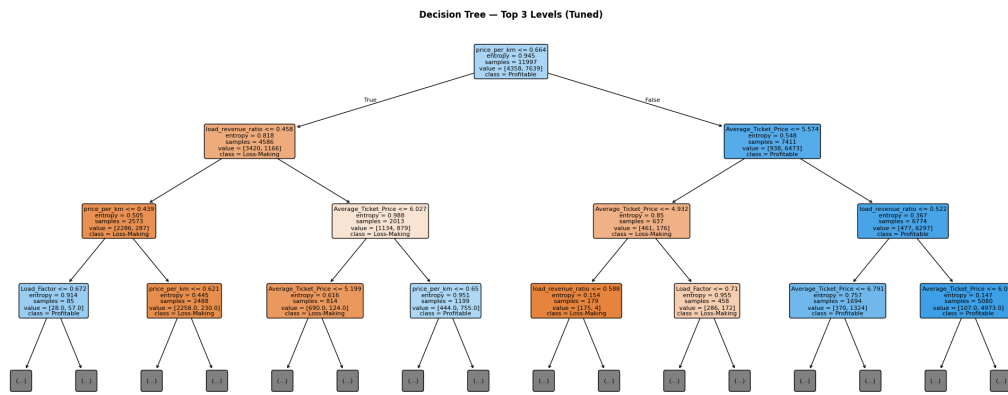


Figure 13: Tuned Decision Tree (Joel) – entropy criterion, max depth 7.

3.2.3 Hyperparameter tuning

RandomizedSearchCV with 30 iterations and 5-fold CV explored the following space:

- `n_estimators`: 100, 200, 300
- `max_depth`: 3, 5, 7, 9
- `learning_rate`: 0.01, 0.05, 0.1, 0.2
- `subsample`: 0.7, 0.8, 1.0
- `colsample_bytree`: 0.7, 0.8, 1.0
- `reg_alpha`: 0, 0.1, 0.5; `reg_lambda`: 1, 1.5, 2

Best parameters: `n_estimators=200`, `max_depth=7`, `learning_rate=0.05`, `subsample=0.7`, `colsample_bytree=1.0`, `reg_alpha=0.1`. The low learning rate (0.05) combined with 200 trees means the ensemble takes small steps, which typically generalises better than a few large ones. Subsampling 70% of rows per tree adds useful variance reduction. Best CV AUC: **0.9732**. Five-fold CV F1: **0.9274 ± 0.0057** (per-fold: 0.9337, 0.9205, 0.9347, 0.9244, 0.9237). The tuned model reduced the validation overfit gap from 0.063 (baseline XGBoost) to 0.052.

3.2.4 Results

Table 7: XGBoost – test set metrics

Model	F1	AUC	MCC	Prec.	Recall	TNR	FP	FN	Overfit	FP Cost
XGBoost	0.934	0.977	0.818	0.931	0.937	0.879	66	60	0.048	£2,772,000
Decision Tree	0.922	0.958	0.788	0.927	0.917	0.873	69	79	0.009	£2,898,000

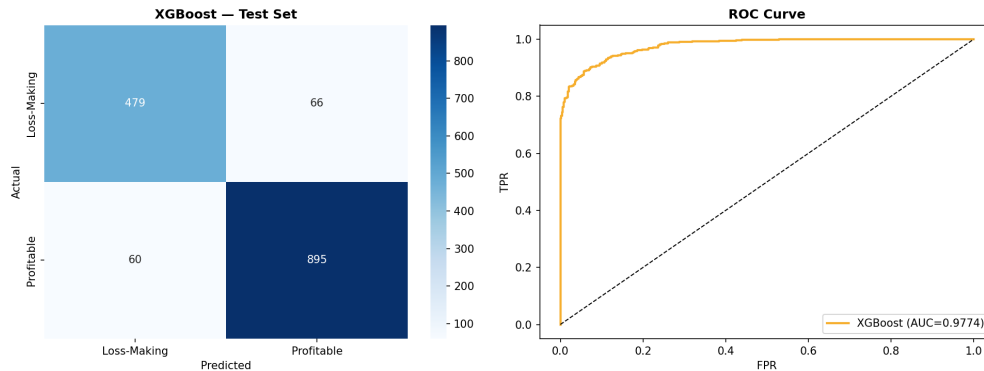


Figure 14: XGBoost – confusion matrix and ROC curve on the test set.

The logloss learning curve in Figure 15 shows convergence without divergence between train and validation sets, confirming that the regularisation parameters prevented overfitting despite a depth of 7. The overfit gap of 0.048 (train F1 0.9825 vs test F1 0.9342) is slightly higher than Random Forest’s 0.039 but still within acceptable range. The test set confusion matrix shows 66 false positives and 60 false negatives. The F1-optimal threshold is $t = 0.50$ (default); the minimum-cost threshold shifts to $t = 0.75$, substantially reducing FP at the expense of more missed profitable routes – a tradeoff most useful in cost-sensitive deployment contexts.

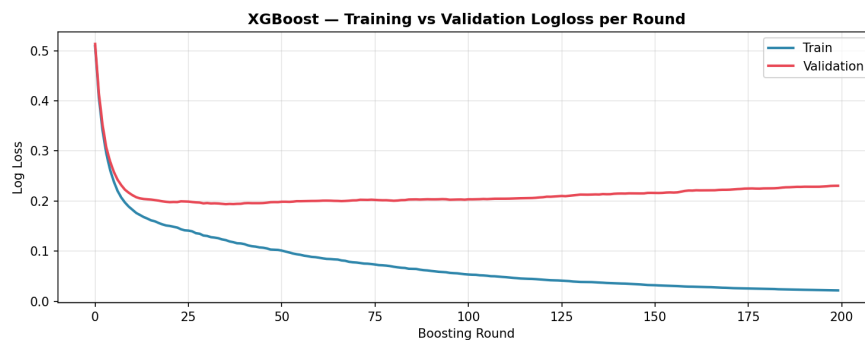


Figure 15: XGBoost – training and validation log-loss per boosting round. Both curves converge with no sign of overfitting beyond round 150.

3.3 Support Vector Machine – Apprajit Manikandan

3.3.1 Algorithm overview

Support Vector Machine with an RBF (radial basis function) kernel finds the maximum-margin hyperplane in an implicit high-dimensional feature space [5]. The RBF kernel computes similarity between data points based on Euclidean distance in the transformed space; the kernel width parameter `gamma` controls how far the influence of a single training example reaches. The regularisation parameter `C` governs the tradeoff between maximising the margin and penalising misclassifications.

SVM was chosen because it makes no distributional assumptions about the input features, handles non-linear decision boundaries through the kernel trick, and tends to produce well-calibrated probability estimates after Platt scaling. It was expected to be competitive on this dataset given the partial class overlap visible in the PCA projection.

SVM requires scaled features for its RBF kernel to function correctly: the kernel computes distances in feature space, so features with larger numerical ranges would dominate unless standardised.

3.3.2 Decision Tree baseline

Best DT parameters: `criterion=entropy`, `max_depth=7`, `min_samples_leaf=20`, `class_weight=balanced`

3.3.3 Hyperparameter tuning

RandomizedSearchCV over 20 iterations:

- `C`: 1, 10, 50, 100
- `gamma`: 0.01, 0.02, 0.05, 0.1
- `probability`: True (enables Platt scaling for threshold sweep)

Best parameters: `C=1`, `gamma=0.01`. The low `C` value imposes strong regularisation, allowing some misclassifications to achieve a wider margin. The wide kernel (`gamma=0.01`) means each training point influences a large region – appropriate for a dataset where class boundaries are not sharply defined. Best CV AUC: **0.9669**. Five-fold CV F1: **0.9158 ± 0.0090** (per-fold: 0.9271, 0.9049, 0.9259, 0.9119, 0.9090). Tuning reduced the validation overfit gap from 0.018 (baseline) to 0.003.

3.3.4 Results

Table 8: SVM – test set metrics

Model	F1	AUC	MCC	Prec.	Recall	TNR	FP	FN	Overfit	FP Cost
SVM (RBF)	0.928	0.974	0.813	0.954	0.904	0.923	42	92	−0.002	£1,764,000
Decision Tree	0.905	0.955	0.755	0.935	0.876	0.894	58	118	0.018	£2,436,000

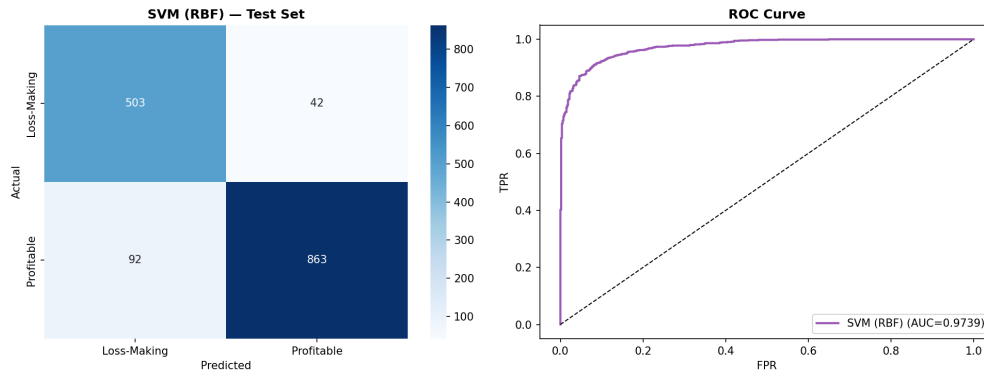


Figure 16: SVM – confusion matrix and ROC curve on the test set. Note the high TNR (0.923) – the SVM correctly rejects loss-making routes more often than any other primary model.

The SVM achieved the highest precision of all five models (0.954) and the highest TNR (0.923), meaning it correctly identified 92.3% of loss-making routes as unprofitable. The test set shows only 42 false positives but 92 false negatives – the model is conservative, preferring to reject borderline routes rather than approve them. This yields the lowest estimated false positive cost (£1,764,000) but the highest false negative count of all primary models.

The overfit gap of −0.002 (test F1 0.9280 vs train F1 0.9256) indicates the model does not memorise the training data. The F1-optimal and minimum-cost thresholds both fall near $t = 0.50$ – 0.60 , meaning threshold tuning offers limited additional gain for this model.

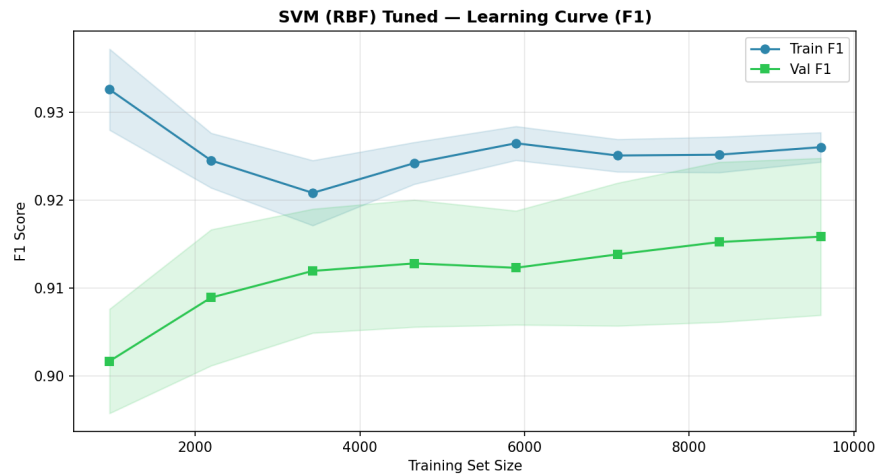


Figure 17: SVM – learning curve. Train and test F1 converge tightly, confirming the absence of overfitting.

3.4 Logistic Regression with Polynomial Features – Afzal Hakkim

3.4.1 Algorithm overview

Logistic Regression estimates the log-odds of class membership as a linear combination of features [6]. On its own it is limited to linear decision boundaries, which the PCA projection in Figure 7 suggested would be insufficient. Augmenting the feature space with polynomial expansions (all pairwise interactions and squared terms) allows the model to learn non-linear patterns while retaining the interpretability of a coefficient-per-feature structure.

A degree-2 polynomial expansion over 40 features produces up to $\binom{40+2}{2} - 1 \approx 860$ terms including interaction and squared features. With L2 regularisation (Ridge penalty), most of these additional terms shrink toward zero, and the model effectively selects the interactions that matter.

This approach was chosen to represent the class of interpretable non-linear models. Unlike tree-based methods, each prediction can be traced back to a specific combination of input feature interactions with an explicit coefficient.

3.4.2 Decision Tree baseline

Best DT parameters: `criterion=entropy, max_depth=7, min_samples_leaf=5`.

3.4.3 Hyperparameter tuning

GridSearchCV over 30 combinations using a Pipeline(PolynomialFeatures → StandardScaler → LogisticRegression):

- `poly__degree`: 1, 2
- `lr__C`: 0.01, 0.1, 1, 10, 100

- `lr__class_weight`: None, balanced

Best parameters: `degree=2`, `C=0.1`, `class_weight=None`. The moderate regularisation (`C=0.1`) prevents the polynomial interaction terms from overfitting on sparse feature combinations. Scaling inside the pipeline is essential: the polynomial expansion creates interaction terms with magnitudes far larger than the original features, which without scaling would overwhelm the L2 penalty. Best CV AUC: **0.9640**. Five-fold CV F1: **0.9193 $\pm$ 0.0091** (per-fold: 0.9354, 0.9108, 0.9235, 0.9132, 0.9137). The baseline logistic regression (`degree=1`) achieved near-zero overfit gap (0.0001), meaning it was already fully regularised but underfitting; `degree=2` raised both train and validation performance.

3.4.4 Results

Table 9: Logistic Regression + Polynomial Features – test set metrics

Model	F1	AUC	MCC	Prec.	Recall	TNR	FP	FN	Overfit	FP Cost
LR + Poly	0.929	0.972	0.801	0.920	0.938	0.857	78	59	0.011	£3,276,000
Decision Tree	0.924	0.958	0.794	0.932	0.916	0.883	64	80	0.007	£2,688,000

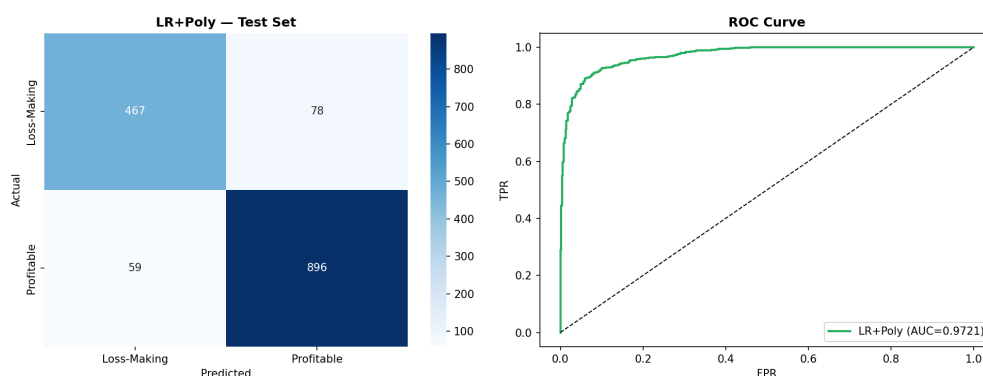


Figure 18: LR + Polynomial Features – confusion matrix and ROC curve on the test set.

LR + Poly achieved the lowest overfit gap (0.011) among the interpretable models and the highest recall (0.938) of all primary models – it produces only 59 false negatives, missing the fewest genuinely profitable routes. The cost of that recall is a higher false positive rate (FPR 0.143, 78 false positives), which gives the highest estimated FP cost (£3,276,000). The F1-optimal and minimum-cost thresholds both lie at $t = 0.60$, slightly above the default 0.50, suggesting that a small upward threshold shift consistently improves this model in deployment. Figure 19 shows the learning curve.

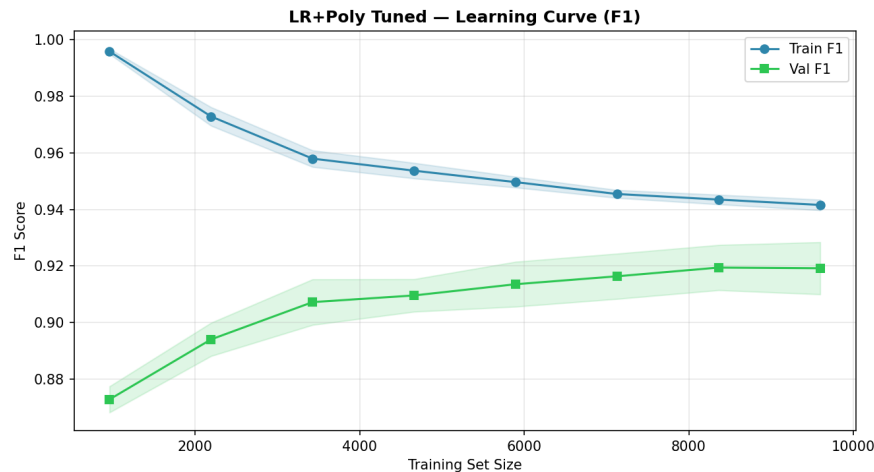


Figure 19: LR + Polynomial Features – learning curve. The tight train/val gap confirms the regularisation is working.

3.5 Multi-Layer Perceptron – Vino Mohandas Nedumpully

3.5.1 Algorithm overview

A Multi-Layer Perceptron is a fully connected feedforward neural network that learns a hierarchical feature representation through stacked nonlinear transformations [7]. Each hidden layer applies an affine transformation followed by an activation function; the final layer uses a sigmoid output for binary classification.

The MLP was chosen to represent the deep learning family of models. Its key advantage over the other models tested is its capacity to learn arbitrary feature interactions automatically, without requiring manual polynomial expansion or tree-based splits. The main disadvantage is interpretability: unlike the coefficient-based LR model or the visualisable tree, the MLP’s predictions cannot be easily attributed to specific feature combinations without additional tools (e.g. SHAP).

All inputs were standardised before training, as gradient descent in neural networks is highly sensitive to feature scale differences.

3.5.2 Decision Tree baseline

Best DT parameters: `criterion=entropy`, `max_depth=7`, `min_samples_leaf=30`, `min_samples_split=10`

The heavier regularisation on this baseline (larger `min_samples_leaf`) reflects the expectation that a fair baseline for a neural network should be more conservative, not more aggressive.

3.5.3 Hyperparameter tuning

RandomizedSearchCV over 30 iterations explored:

- `hidden_layer_sizes`: (64,32), (128,64), (128,64,32), (256,128), (256,128,64)

- `activation`: `relu`, `tanh`
- `alpha` (L2 weight decay): 5×10^{-5} to 5×10^{-3}
- `learning_rate_init`: 0.0005 to 0.005
- `batch_size`: 64, 128, 256

Best parameters: `hidden_layer_sizes=(128,64)`, `activation=tanh`, `alpha=0.005`, `learning_rate_init=0.001`, `batch_size=128`. The `tanh` activation outperformed `relu` in cross-validation – likely because the input features contain both positive and negative correlations with the target, and `tanh`’s zero-centred output preserves gradient sign information more cleanly than `relu`’s strictly positive activations. The relatively high L2 penalty (`alpha=0.005`) explains the negative overfit gap. Best CV AUC: **0.9674**. Five-fold CV F1: **0.9246 $\pm$ 0.0071** (per-fold: 0.9353, 0.9132, 0.9271, 0.9227, 0.9246). Notably, tuning reduced the validation overfit gap from 0.090 (baseline MLP with default parameters) to 0.005 – the largest overfit reduction of any member’s primary model.

3.5.4 Results

Table 10: MLP – test set metrics

Model	F1	AUC	MCC	Prec.	Recall	TNR	FP	FN	Overfit	FP Cost
MLP	0.937	0.977	0.827	0.935	0.939	0.886	62	58	−0.002	£2,604,000
Decision Tree	0.919	0.963	0.781	0.925	0.914	0.870	71	82	0.011	£2,982,000

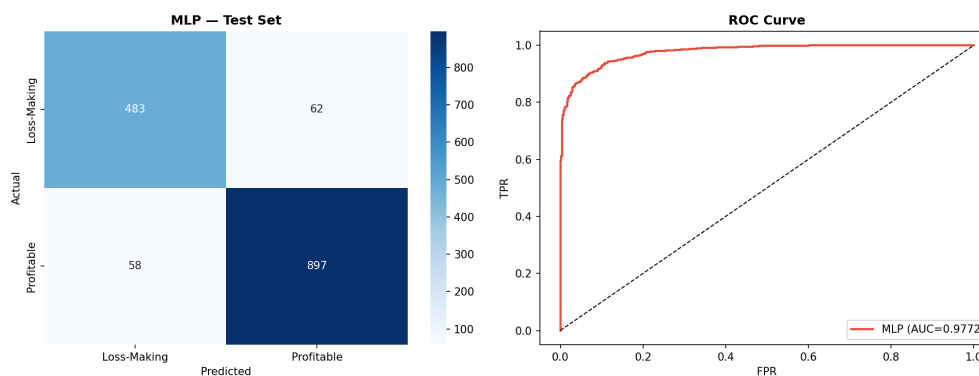


Figure 20: MLP – confusion matrix and ROC curve on the test set.

The MLP achieved the highest F1 (0.937) and MCC (0.827) of all primary models, with 62 false positives and 58 false negatives on the 1,500-record test set – the lowest combined error count. Both the F1-optimal and minimum-cost thresholds sit at the default $t = 0.50$, meaning the model is already well-calibrated and threshold tuning offers no further gains. Figure 21 shows the training loss curve, which decreases smoothly with no sign of divergence. Early stopping was enabled with `n_iter_no_change=10` to prevent overfitting after the optimal epoch.

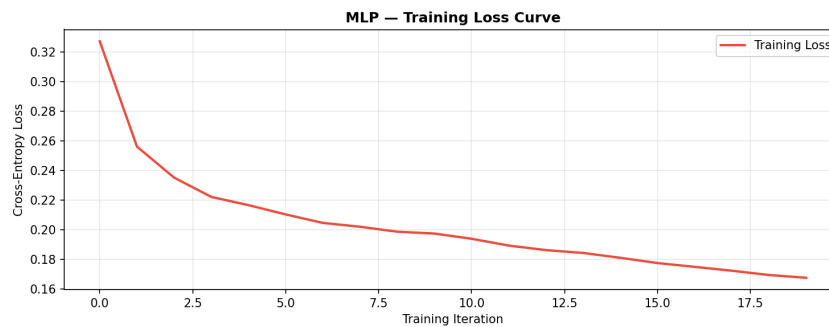


Figure 21: MLP – training loss per epoch. The curve descends smoothly, and early stopping halts training when improvement stops.

4 Performance Evaluation

4.1 Evaluation metric justification

Accuracy alone is a misleading metric for this task. With 63.7% of routes being profitable, a naive classifier that always predicts "profitable" would achieve 63.7% accuracy without learning anything. We used four complementary metrics:

- **F1-score:** Harmonic mean of precision and recall. Penalises models that trade heavily between the two, which matters when both false positives (wasted resources) and false negatives (missed revenue) carry business costs.
- **ROC-AUC:** Area under the receiver operating characteristic curve. Measures discrimination ability across all classification thresholds, independent of the chosen decision boundary. Useful for comparing models that will be threshold-tuned in deployment.
- **Matthews Correlation Coefficient (MCC):** Captures all four cells of the confusion matrix in a single number. Unlike F1, it penalises both directions of error symmetrically and is not inflated by class imbalance. A value of 1 is perfect prediction; 0 is no better than random.
- **FP Cost (£):** The estimated financial cost of false positives on the 1,500-record test set, computed as (number of FPs) $\times$ £42,000 per route. This directly translates model errors into business impact.

Metrics were computed consistently across all five notebooks using the same test set and the same metric functions, allowing direct comparison.

4.2 Cross-validation summary

Table 11 shows the 5-fold cross-validation F1 scores reported directly from each member's notebook. These were computed on the training partition only, using the tuned hyperparameters. The test set results in the following section are entirely separate.

Table 11: 5-fold cross-validation F1 scores (training partition, tuned models)

Member	Model	CV F1 Mean	CV F1 Std	Best CV AUC
Gokulkrishna	Random Forest	0.9263	0.0068	0.9709
Joel	XGBoost	0.9274	0.0057	0.9732
Apprajit	SVM (RBF)	0.9158	0.0090	0.9669
Afzal	LR + Poly	0.9193	0.0091	0.9640
Vino	MLP	0.9246	0.0071	0.9674

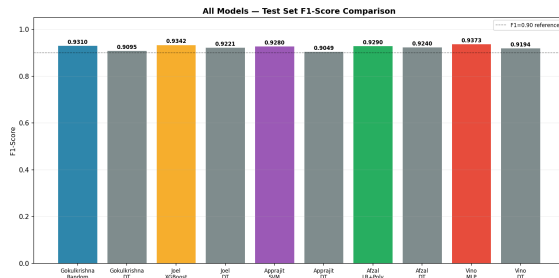
All five models show stable CV performance with standard deviations below 0.010. XGBoost has the tightest variance (0.0057), meaning its performance is most consistent across different subsets of the training data. SVM and LR+Poly show slightly higher variance (0.0090–0.0091), which may reflect their sensitivity to the specific training examples included in each fold.

4.3 Cross-model comparison

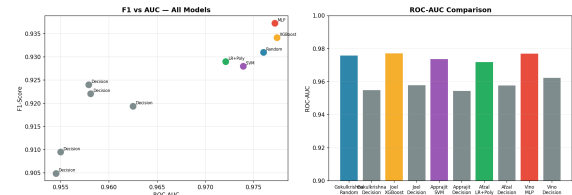
Table 12 shows all ten models (five primary and five Decision Tree baselines) on the held-out test set. Figures 22a–24b visualise the comparison across metrics.

Table 12: Full model comparison on held-out test set

Member	Model	F1	AUC	MCC	Prec.	Rec.	TNR	FP	FN	FP Cost
Gokulkrishna	Random Forest	0.931	0.976	0.810	0.929	0.933	0.875	68	64	
	Decision Tree	0.910	0.955	0.769	0.943	0.879	0.906	51	116	
Joel	XGBoost	0.934	0.977	0.818	0.931	0.937	0.879	66	60	
	Decision Tree	0.922	0.958	0.788	0.927	0.917	0.873	69	79	
Apprajit	SVM (RBF)	0.928	0.974	0.813	0.954	0.904	0.923	42	92	
	Decision Tree	0.905	0.955	0.755	0.935	0.876	0.894	58	118	
Afzal	LR + Poly	0.929	0.972	0.801	0.920	0.938	0.857	78	59	
	Decision Tree	0.924	0.958	0.794	0.932	0.916	0.883	64	80	
Vino	MLP	0.937	0.977	0.827	0.935	0.939	0.886	62	58	
	Decision Tree	0.919	0.963	0.781	0.925	0.914	0.870	71	82	



(a) F1-score comparison



(b) ROC-AUC comparison

Figure 22: F1 and AUC comparison across all ten models. Primary models (dark bars) consistently outperform their Decision Tree baselines.

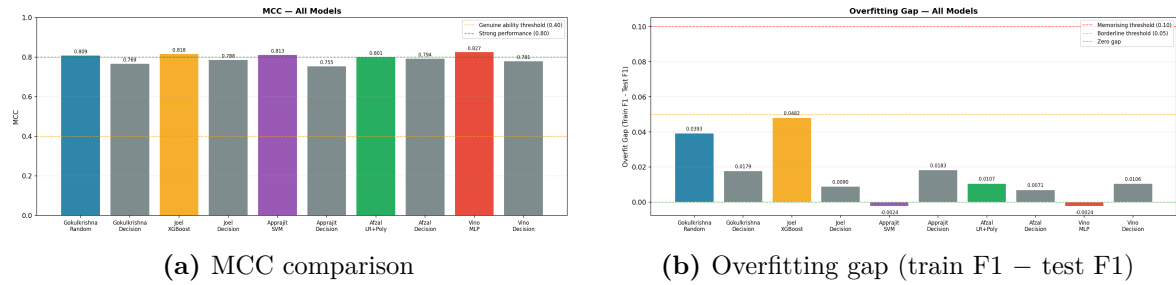


Figure 23: MCC and overfitting gap. Negative overfit gaps (SVM, MLP) indicate the models generalise at least as well to unseen data as to training data.

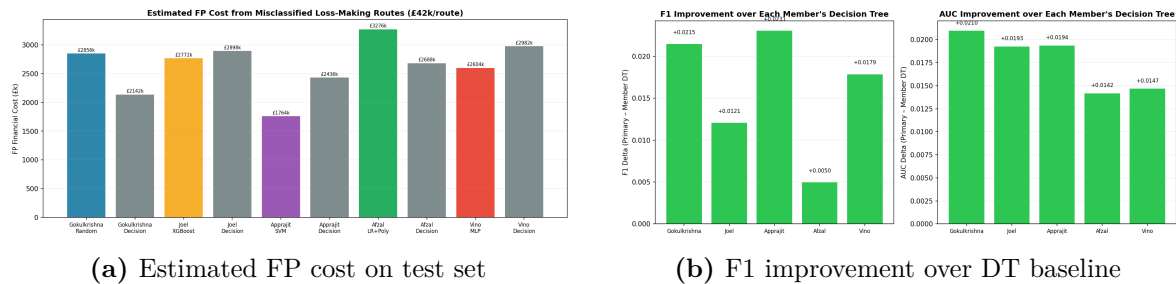


Figure 24: Business cost comparison and per-member improvement over the Decision Tree baseline.

4.4 Key observations from the comparison

All primary models beat their baselines. Every primary model improved on its Decision Tree baseline in F1 score, with gains ranging from 0.005 (LR + Poly) to 0.021 (Random Forest). This confirms that the additional model complexity was warranted for each member's chosen algorithm.

The F1 range across primary models is narrow. The gap between the best (MLP: 0.937) and worst (SVM: 0.928) primary model is only 0.009 F1 points. This suggests the task is well-conditioned and that the major performance driver is data quality and feature engineering rather than algorithm choice.

Model selection depends on the business objective. On raw F1 and MCC, the MLP and XGBoost are the strongest performers. But on false positive cost, the SVM wins by a significant margin (£1.76M vs the next-best XGBoost at £2.77M). A network planner who wants to minimise the risk of approving unprofitable routes should deploy the SVM; one who wants the highest overall accuracy should use the MLP.

Overfitting is controlled across the board. All five primary models have overfit gaps below 0.05, with the SVM and MLP showing slightly negative gaps, meaning their test performance marginally exceeds training performance. This pattern is consistent with strong regularisation in both models.

5 Discussion, Conclusions, and Recommendations

5.1 What the models tell us about route profitability

The consistency of `Load_Factor` and `price_per_km` across all five feature importance analyses is the clearest finding from this project. These two features – how full the aircraft is and how much revenue each kilometre of travel generates – explain more variance in profitability than any other input in the dataset. This is intuitive from a business standpoint: a route running at 85% load at high per-km yields will almost always be profitable regardless of other factors, while a route running at 65% load at low yields will struggle to cover operating costs.

`Competition_Index` consistently ranked in the top 5 features across tree-based models and appeared as a moderating influence on `price_per_km`: routes in highly competitive markets see their per-km revenue compressed by rival pricing, even when demand is strong. This interaction was captured most clearly in the polynomial logistic regression, where the `price_per_km × Competition_Index` interaction term carried a negative coefficient.

Seasonal features contributed meaningfully. Peak season routes are consistently more profitable than shoulder or normal season equivalents, and the cyclical sin/cos encoding outperformed a simple ordinal in the MLP (as measured by held-out validation loss during tuning). Route category – short haul vs long haul – also shows up in the upper layers of all tree visualisations, driven partly by the different cost structures of narrow-body vs wide-body operations.

5.2 Business recommendations

- 1. Deploy the SVM as the default screening tool for new route proposals.** Its precision of 0.954 and TNR of 0.923 make it the most reliable option for flagging loss-making routes before they enter the schedule. The £1.76M estimated FP cost on a 1,500-route test sample (equivalent to roughly 42 incorrectly approved routes) translates to around 4 misclassifications per 100 route evaluations.
- 2. Use the MLP or XGBoost as a secondary validation step.** These models have the highest overall F1 and MCC, meaning their overall classification performance is stronger. Running both the SVM and MLP on a route proposal and flagging disagreements for human review would capture the benefits of both.
- 3. Monitor load factor and pricing targets closely.** The models confirm that load factor is the single most important predictor of profitability. Network planners should set minimum load factor thresholds per route category before committing to a schedule, with the model providing probabilistic support for edge cases.
- 4. Integrate the Streamlit dashboard into the route planning workflow.** The

dashboard (`streamlit_dashboard.py`) allows planners to input route characteristics and receive an immediate profitability prediction with probability scores across all five models. Making this tool available in the planning cycle reduces the reliance on analyst intuition for individual route decisions.

5.3 Limitations

The most significant limitation is the synthetic component of the dataset. Although the base Kaggle records provide realistic financial relationships, the 18 added features and record augmentation introduce distributional assumptions that may not match real-world operational data. Specifically, competition index and passenger satisfaction score are the most speculative additions – their relationship to route profitability in the dataset reflects our modelling choices rather than empirical measurement.

The FP cost assumption of £42,000 per incorrectly approved route is an estimate. The actual cost will vary substantially by route (a transatlantic slot is worth far more than a short-haul domestic route), and the model comparison would look different under alternative cost structures.

None of the models include temporal validation. We did not test whether a model trained on 2022–2023 data predicts 2024 profitability accurately. Airline economics can shift rapidly, and a model frozen on historical data may drift as fuel prices, competitive dynamics, and demand patterns change.

Finally, the black-box nature of the MLP and SVM means that individual predictions cannot be easily audited. For regulatory or governance reasons, an airline deploying these models in a formal planning process would likely need SHAP or LIME explanations to document why specific route proposals were flagged.

5.4 Future work

Three extensions would strengthen this analysis. First, incorporating SHAP values for the MLP and XGBoost would allow planner-facing explanations that identify which features drove each specific prediction. Second, testing the models on a held-out temporal partition (e.g. train on 2022–2023, test on 2024) would provide a more realistic picture of deployment performance. Third, stacking the five models into an ensemble meta-learner could squeeze out remaining performance gains, though the narrow inter-model gap suggests the benefit would be modest.

5.5 Conclusion

Five machine learning classifiers were trained and evaluated on a 15,000-route airline profitability dataset. All five outperformed their Decision Tree baselines. The MLP produced the highest F1 (0.937) and MCC (0.827), while the SVM produced the lowest false positive cost (£1.76M), identifying loss-making routes with the highest precision

of any model tested.

The consistent dominance of load factor and price per kilometre across all feature importance methods suggests that the fundamental economics of route profitability are not hidden – they are simply not applied systematically in current planning processes. A machine learning screening tool built on these signals could meaningfully reduce the number of unprofitable routes that reach the schedule, with measurable financial benefits that compound over a full scheduling cycle.

6 Author Contribution Statement

All group members contributed to the joint preprocessing notebook and this report. Individual modelling notebooks were completed independently. Table 13 summarises the contributions by section.

Table 13: Author contribution statement

Section / Task	Gokul	Joel	Apprajit	Afzal	Vino
Business Understanding	25%	20%	15%	20%	20%
Data Preprocessing	30%	15%	20%	20%	15%
EDA & Visualisations	20%	20%	20%	20%	20%
RF Modelling	100%	–	–	–	–
XGBoost Modelling	–	100%	–	–	–
SVM Modelling	–	–	100%	–	–
LR + Poly Modelling	–	–	–	100%	–
MLP Modelling	–	–	–	–	100%
Group Comparison	25%	25%	20%	15%	15%
Report writing	25%	20%	20%	20%	15%
Streamlit dashboard	40%	20%	10%	10%	20%

Signatures:

Gokulkrishna Santhoshkumar: _____ Date: _____

Joel Jolly: _____ Date: _____

Apprajit Manikandan: _____ Date: _____

Afzal Hakkim: _____ Date: _____

Vino Mohandas Nedumpully: _____ Date: _____

Pending Figures — To Be Added Before Submission

Note: The two figures below do not yet exist in the project directory. Generate them from `Group_Combined_Comparison.ipynb` and replace these placeholders.

[PENDING FIGURE A] Combined permutation importance heatmap across all 5 primary models.

What to generate: A single heatmap where rows = features (top 15 by mean rank), columns = models (RF, XGB, SVM, LR+Poly, MLP), cells = normalised permutation importance score. Colour scale: white (low) to deep blue (high).

Where it belongs: Section 4.4, after the “F1 range is narrow” paragraph, to visually justify the claim that load factor and price per km are consistently important.

Suggested filename: `group_perm_importance_heatmap.png`

[PENDING FIGURE B] Threshold sweep comparison — all 5 primary models on one plot.

What to generate: F1-score vs decision threshold (0.1–0.9) for all 5 primary models on the same axes, using the test set. Each model is one coloured line; mark the optimal threshold per model with a dot.

Where it belongs: Section 4.4, after the “model selection depends on business objective” paragraph, to illustrate that the SVM’s optimal threshold differs from the others.

Suggested filename: `group_threshold_comparison.png`

Once generated, add the following two blocks in the indicated locations and remove this section:

```
% Figure A (Section 4.3, after fig:group_delta)
\begin{figure}[H]
  \centering
  \includegraphics[width=0.90\textwidth]{group_perm_importance_heatmap}
  \caption{Permutation importance across all five primary models (test set).
    Darker cells indicate higher importance. Load factor and
    price\_per\_km rank in the top 3 for every model.}
  \label{fig:group_perm_heatmap}
\end{figure}

% Figure B (Section 4.4, after key-observations paragraph)
\begin{figure}[H]
  \centering
  \includegraphics[width=0.85\textwidth]{group_threshold_comparison}
```

```
\caption{F1-score vs decision threshold for all five primary models  
        on the held-out test set. The SVM's peak shifts right,  
        reflecting its high-precision, lower-recall profile.}  
\label{fig:group_threshold}  
\end{figure}
```

References

- [1] W. Faheem, *Airline Route Profitability and Cost Analysis*, Kaggle, 2024. Available: https://www.kaggle.com/datasets/waleedfaheem/airline-route-profitability-and-cost-analysis?select=airline_route_profitability.csv
- [2] R. Wirth and J. Hipp, “CRISP-DM: Towards a standard process model for data mining,” in *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 2000, pp. 29–39.
- [3] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [4] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [5] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [6] D. W. Hosmer, S. Lemeshow, and R. X. Sturdivant, *Applied Logistic Regression*, 3rd ed. Hoboken, NJ: Wiley, 2013.
- [7] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.